



WEEK 2 • MLP CLASSIFICATION

從 2D 到 784D

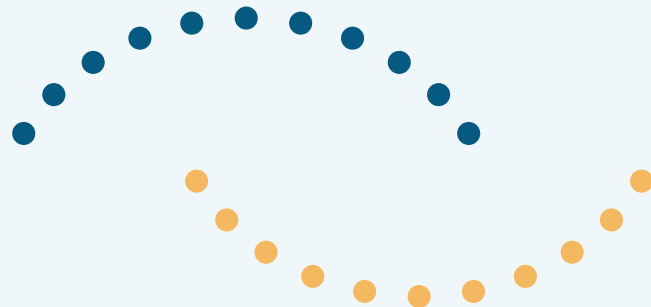
同一把尺，切開不同維度的空間

從 make_moons 的 2D 分類，到 MNIST 手寫數字辨識

上週學到的事

- part2：2D 資料（make_moons），兩群交纏的月牙形
- 純線性模型切不開 → 加入 ReLU + 隱藏層後，可以完美分類
- 今天要做的事：把同一套架構搬到 MNIST（784 維、10 個類別）

make_moons：兩群交纏的資料



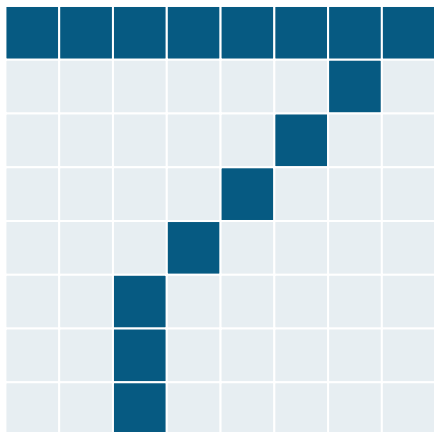
直線切不開，需要非線性邊界

不管輸入是 2 個數字，還是 784 個數字，
分類問題的本質完全相同：

在一個多維空間中，找出一個邊界，
把不同類別的點群分開。

2D 我們畫得出這條線； 784D 畫不出來，但數學上是同一件事。

MNIST 資料表示法



28 × 28 灰階圖片



[0, 0, 0.2, 0.9, ... ,
0.1, 0]
共 784 個數字

攤平成 784 維向量

- 每張圖是 28×28 灰階像素
- 攤平成 784 維向量
- 每張圖 = 784 維空間中的一個點
- 10 個類別（數字 0-9）= 10 群點

架構完全對稱

part2_mlp_classification.py (moons)

```
nn.Linear(2, 16)
```

```
nn.ReLU()
```

```
nn.Linear(16, 2)
```

part3_mlp_mnist.py (MNIST)

```
nn.Linear(784, 128)
```

```
nn.ReLU()
```

```
nn.Linear(128, 10)
```

訓練迴圈、 *loss function* (*CrossEntropyLoss*) 完全沒變 —— 只有輸入 / 輸出的維度變了。

● RESULT

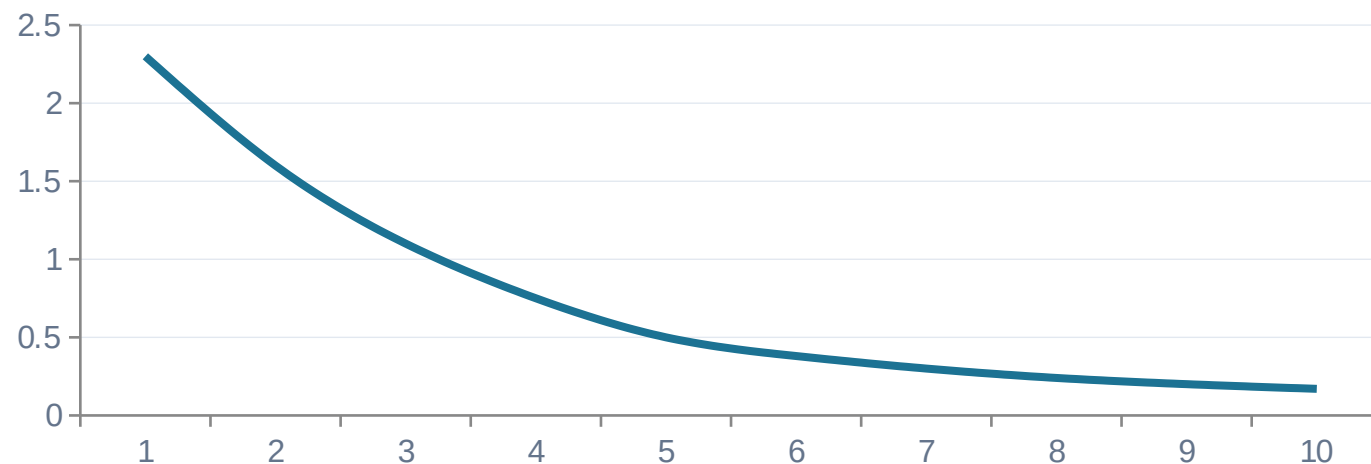
訓練結果 (含 ReLU)

97-98%

test accuracy

`Linear(784,128) → ReLU → Linear(128,10)`

Training Loss (示意)



同一套「切邊界」的邏輯，維度變高，程式碼幾乎不用改。

一個實驗：拿掉 ReLU 會怎樣？

```
nn.Linear(784, 128)
```

```
nn.Linear(128, 10) ← ReLU 被拿掉了
```

兩個線性層疊在一起，沒有非線性夾在中間，數學上等價於：

$$W_2(W_1x + b_1) + b_2 = W'x + b' \quad (\text{還是一個線性函數})$$



結果揭曉

含 ReLU (part3)



拿掉 ReLU (part3b)



差距只有幾個百分點，遠比想像中小。

對比 `demo_linear_limits.py` :

拿掉非線性在 sine wave 上是災難性失敗（直線切不出曲線）；
但在 MNIST 上，準確率幾乎沒差。

為什麼？

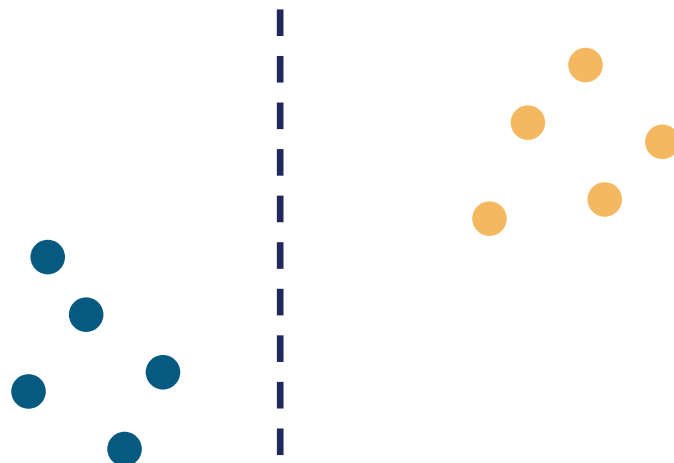
關鍵概念：線性可分

$$f(x) = wx + b$$

只靠這個線性函數的分數排序，就能把各類別正確分開

- 定義：若存在一個超平面（hyperplane），能把各類別（近乎）完全分開，這個資料集就稱為線性可分
- MNIST 的每個數字類別，在 784 維空間裡，剛好幾乎滿足這個條件
- 但這不是每個資料集都有的性質 —— make_moons 就沒有

線性可分（示意）



為什麼高維空間更容易線性可分

1

784 個自由度

784 個權重遠比 2D 直線的「斜率+截距」兩個自由度更有彈性

2

更容易找到分界超平面

自由度越多，越容易在空間裡找到一個超平面把 10 群點隔開

3

moons 是例外

moons 的形狀在 2D 空間中彼此糾纏，不管維度高低，直線都切不開這種纏繞結構

補充：這跟 VC dimension / 模型容量 (capacity) 的概念有關。

那 ReLU 還有沒有用？

+6~8%

有！ 97-98% vs 90-92%
多出的幾個百分點就是 ReLU 的貢獻

- 非線性負責處理「幾乎線性可分，但還差一點」的部分
- 典型案例：筆畫相近的數字，例如 4 vs 9、3 vs 5 vs 8
- 這些邊界案例需要彎曲的決策邊界才能正確分開

怎麼驗證「幾乎線性可分」？

問題

784 維畫不出來，怎麼眼見為憑？

part2 可以直接畫決策邊界，因為 input 是 2D；MNIST 是 784D，沒辦法直接畫。



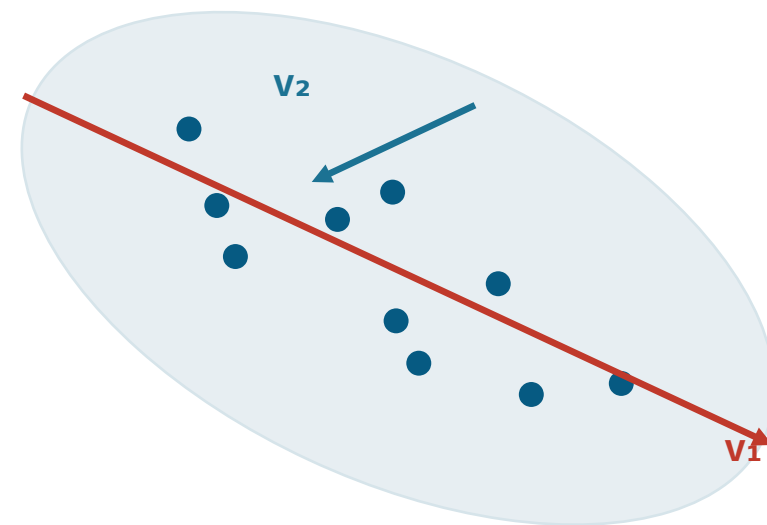
答案

PCA 降維

把 784 維投影到 2D —— 只是為了「看」，不是為了訓練模型。

PCA 數學核心

- 目標：找到資料變化最大的方向，投影到低維時盡量保留這些變化
- 作法：找共變異數矩陣的前兩大特徵向量 v_1, v_2
- 投影： $z = (v_1^T x, v_2^T x)$ — 把 784 維的 x 壓成 2 個數字



一句話：PCA 是把「資料鋪開最廣」的那兩個方向抓出來。

PCA 驗證：原始像素空間

Raw Pixel Space (PCA)



- 把測試集的 784 維像素直接投影到 2D
- 10 種顏色代表數字 0-9
- 雖有些重疊（4/9、3/5/8 附近較亂），整體已有明顯分群趨勢
- 這就是「線性模型能拿 90%+」背後看得到的證據

PCA 驗證： Hidden Layer 表示法

Hidden Layer Representation (PCA)



- 同一批資料丟進含 ReLU 的 MLP，取 hidden layer 128 維輸出
- 再用 PCA 降到 2D，同樣的著色方式
- 群更緊、分得更開
- 這就是那多出來的 6-8 個百分點從哪裡來的 —— 非線性把糾纏的部分攤開了

這堂課的三個重點

1 本質不變

分類 = 在某個維度空間中切邊界。從 2D 的 moons 到 784D 的 MNIST，架構與訓練迴圈幾乎不用改。

2 高維更容易線性可分

高維空間比想像中容易線性可分，MNIST 恰好幾乎滿足這個性質，這也是拿掉 ReLU 準確率沒暴跌的原因。

3 非線性補上最後一段

ReLU / 非線性負責處理剩下那一小段「切不乾淨」的邊界，把 90%+ 推向 97-98%。

4 PCA 是我們的眼睛

PCA 讓我們「偷看」高維空間發生什麼事——驗證了資料在被 MLP 重新排列後變得更容易分開。